

Reviewer Pack — Minimal Geometric Entropy of Tautological Ideals and Circuit...

Entry db70af57b32c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-11 02:38:37 UTC

Minimal Geometric Entropy of Tautological Ideals and Circuit Depth

Entry ID: db70af57b32c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-11 02:38:37 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory - Entropy of Tautological Ideals **Field B** (complexity object): Boolean Circuits - Circuit Depth

Statement:

For every Boolean circuit C with depth d , the minimal geometric entropy ($H_{\min}$) of the tautological ideal I_C is $\Theta(d^2 \log n)$, where n is the number of variables in C .

Rationale (proposer's reasoning):

The minimal geometric entropy measures the complexity of algebraic varieties and has been studied in Geometric Complexity Theory. The connection to circuit depth suggests that this invariant could provide insights into the structure of Boolean circuits, potentially leading to new lower bounds for circuit complexity.

Taxonomy category: GeometricEntropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d217d5d952f34771

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 95% of the circuits with depth d show minimal geometric entropy ($H_{\min}$) within $\pm 10\%$ of $\Theta(d^2 \log n)$, and a Pearson correlation coefficient of $r \geq 0.9$ between $H_{\min}$ and $d^2 \log n$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric complexity theory" AND "entropy of tautological ideals" AND boolean circuits - "tautological ideal" AND minimal geometric entropy" AND circuit depth" - "Boolean circuit depth" AND "entropies in Geometric Complexity Theory"

Top relevant hits considered: - [s2:83ab0f7b804d184f8af92fe4ba3b0f7217e64be2] Computation and logic in the real world : Third Conference on Computability in Europe, CiE 2007 Siena, Italy, June 18-23 - [s2:62f6a2bb598023fa66600d12da9893d2acfee8a8] Computation and logic in the real world : Third Conference on Computability in Europe, CiE 2007 Siena, Italy, June 18-23

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows = len(matrix)
    if rows == 0:
        return 0
    cols = len(matrix[0])

    rank = 0
    for i in range(rows):
        pivot_row = i
        while pivot_row < rows and matrix[pivot_row][i] == 0:
            pivot_row += 1
        if pivot_row == rows:
            continue

        # Swap rows
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]

        # Eliminate below the pivot
        for j in range(i + 1, rows):
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(cols):
                if i == k:
                    matrix[j][k] = 0
                else:
                    matrix[j][k] += factor * matrix[i][k]
```

```

        rank += 1

    return rank

def generate_boolean_circuit(depth, n):
    circuit = []
    for _ in range(depth):
        gate_type = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, 1) for _ in range(n)]
        circuit.append((gate_type, inputs))
    return circuit

def tautological_ideal(circuit):
    n = len(circuit[0][1])
    variables = list(range(n))
    ideal = []

    def evaluate_circuit(gate, inputs):
        if gate == 'AND':
            return all(inputs)
        elif gate == 'OR':
            return any(inputs)

    for i in range(2**n):
        inputs = [(i >> j) & 1 for j in range(n)]
        result = evaluate_circuit(circuit[0][0], inputs)
        for gate, inputs in circuit[1:]:
            result = evaluate_circuit(gate, [result] + inputs)
        if not result:
            ideal.append(inputs)

    return ideal

def minimal_geometric_entropy(ideal):
    n = len(ideal[0])
    matrix = []
    for row in ideal:
        matrix.append([Fraction(row[i]) for i in range(n)])

    rank = gaussian_elimination(matrix)
    H_min = -rank * math.log2(rank) if rank > 0 else 0
    return H_min

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    total_H_min = 0.0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5): # Sample 5 instances per size
            circuit = generate_boolean_circuit(random.randint(1, n), n)
            ideal = tautological_ideal(circuit)
            H_min = minimal_geometric_entropy(ideal)
            total_H_min += H_min
            instances_tested += 1

```

```

mean_H_min = total_H_min / instances_tested
conjecture_holds = all(abs(H_min - (d**2 * math.log(n))) <= 0.1 * (d**2 * math.log(n)) for d,

return {
    "metric_name": "minimal_geometric_entropy",
    "metric_value": mean_H_min,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)] # Default to first

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_H_min = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_H_min} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_H_min} std=0.0 support_fraction={support_fraction:.2f}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e101adac.py", line 119, in <module>
    result = run_trial(seed)
            ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e101adac.py", line 97, in run_trial
    H_min = minimal_geometric_entropy(ideal)
            ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e101adac.py", line 83, in minimal_geome
    rank = gaussian_elimination(matrix)
            ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e101adac.py", line 27, in gaussian_elim
    while pivot_row < rows and matrix[pivot_row][i] == 0:
            ~~~~~^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14764
2	propose	ollama_remote	glm4:latest	0	0	12463
3	preregistration	ollama_remote	glm4:latest	0	0	14546
4	novelty	ollama_remote	glm4:latest	0	0	8660
5	novelty	ollama_remote	glm4:latest	0	0	10090
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28171
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15559
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19994
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12491
10	judge	ollama_remote	glm4:latest	0	0	11521

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148259 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/db70af57b32c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/db70af57b32c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/db70af57b32c.tar.gz (if generated)